Lecture 2

Binary Arithmetic, Data Representation and Boolean Algebra

GOLDEN GATE UNIVERSITY

Computer Organization and Digital Foundations

Dr. Yatish Bathla, PhD. (Information Technology), Obuda University, Budapest

Recap and Roadmap

A **Binary arithmetic and signed numbers**
Add, subtract, multiply, complements, overflow — Slides 3-9

B **Data representation**
Codes, floating point, memory layout, text, images, sound — Slides 10-13

C **Boolean algebra and logic gates**
Gates, truth tables, laws, canonical forms — Slides 14-19

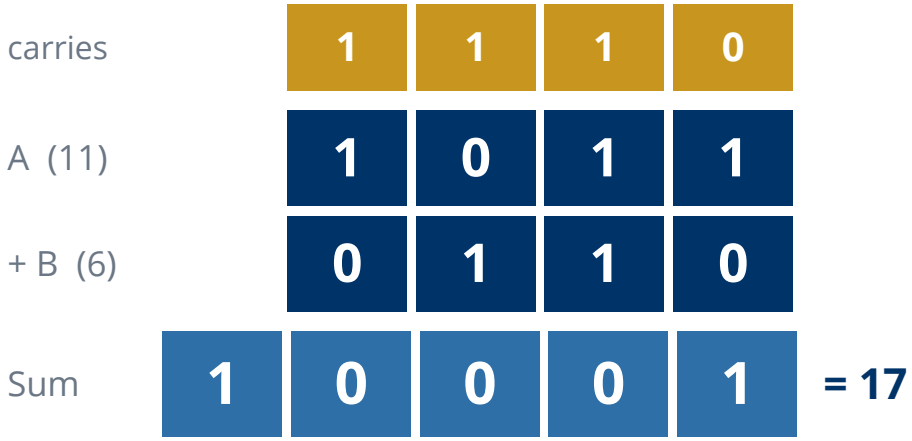
Recap of Hour 1: value = Σ digit × radix^{i}, and every number can be converted between bases.

Binary Addition

A	B	Carry in	Sum	Carry out
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
1	1	1	1	1

Only four cases to learn; the last row shows 1 + 1 + carry-in 1.

Worked example: $1011_2 + 0110_2$



A hardware adder repeats this column rule: sum = A ⊕ B ⊕ carry-in.

Binary Subtraction

A	B	Borrow in	Difference	Borrow out
0	0	0	0	0
1	0	0	1	0
1	1	0	0	0
0	1	0	1	1

The highlighted row needs a borrow: 0 − 1 becomes 10 − 1 = 1.

Worked example: $1101_2 - 0110_2$

borrow	0	1	1	0	
A (13)	1	1	0	1	
− B (6)	0	1	1	0	
Diff	0	1	1	1	= 7

Borrowing is error-prone, so hardware subtracts by adding a complement.

Binary Multiplication and Division

Multiply: $101_2 \times 11_2$ (5 × 3)

Multiplicand		1	0	1	
× Multiplier		1	1		
bit 0 = 1		1	0	1	
bit 1 = 1	1	0	1	0	
Sum	1	1	1	1	= 15

Shifts are fast multiplies

- Left shift by 1 = × 2
- Right shift by 1 = ÷ 2
- 0011 ≪ 2 = 1100 (3 → 12)

Division

Repeated subtract-and-shift, like long division; more complex hardware than multiplication

Multiply = shift and add. Divide = shift and subtract.

Complements

	1's complement	2's complement
Rule	Flip every bit	1's complement + 1
Formula (n bits)	$(2^n - 1) - N$	$2^n - N$
N = 1011	0100	0101
N = 0110	1001	1010
N = 0001	1110	1111

Shortcut

Start at the right. Copy bits up to and including the first 1, then flip the rest.

Check: 1011 + 0101 = 1 0000 → discard the carry → 0000 (so 0101 acts as −1011)

2's complement turns subtraction into addition: A − B = A + (2's complement of B).

Subtraction Using 2's Complement

1 **Complement**
Take the 2's complement of B

2 **Add**
Add it to A

3 **Check carry**
Carry out? Discard it, result is positive. No carry? Negative, in 2's complement form

7 − 5 (4 bits)

A = 0111 B = 0101

2's comp of B = 1011

A + comp = 1 0010

Carry out → discard → 0010 = +2

5 − 7 (4 bits)

A = 0101 B = 0111

2's comp of B = 1001

A + comp = 0 1110

No carry → negative: 2's comp of 1110 = 0010 → −2

Signed Binary Numbers

Value	Sign-magnitude	1's complement	2's complement
+7	0111	0111	0111
+5	0101	0101	0101
+0	0000	0000	0000
−0	1000	1111	(none)
−5	1101	1010	1011
−7	1111	1000	1001
−8	(none)	(none)	1000

Verdict

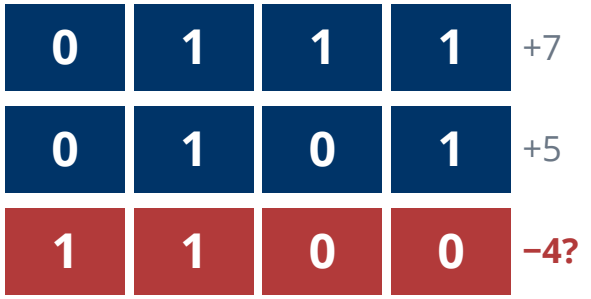
2's complement wins: one zero, one adder, range −8 to +7

Reading 2's complement: the leftmost bit has weight -2^{n-1}. 1011 = −8 + 2 + 1 = −5

Range and Overflow

Bits	Unsigned range	Signed (2's complement)
4	0 to 15	−8 to +7
8	0 to 255	−128 to +127
16	0 to 65,535	−32,768 to +32,767
32	0 to 4,294,967,295	−2,147,483,648 to +2,147,483,647

Overflow example



Overflow rule

Two positives give a negative, or two negatives give a positive: overflow

Why it matters

Choose a data width that fits the largest value your program can produce

Binary Codes: BCD, Gray and ASCII

Dec	Binary	BCD (digit)	Gray (3-bit)
0	000	0000	000
1	001	0001	001
2	010	0010	011
3	011	0011	010
4	100	0100	110
5	101	0101	111
6	110	0110	101
7	111	0111	100

BCD

59 → 0101 1001 (each digit in 4 bits)

Gray code

Neighbours differ in exactly one bit

ASCII

'A' = 65 = 1000001_2 = 41_{16}; '0' = 48

Floating-Point Numbers

IEEE 754 single precision (32 bits)



Worked example: 6.5

$6.5 = 110.1_2 = 1.101_2 \times 2^2$

Sign = 0

Exponent = 2 + 127 = 129 = 10000001_2

Fraction = 101 000 …

Hex: 40D00000

Remember

- Value = $(-1)^s \times 1.\text{fraction} \times 2^{(\text{exp} - \text{bias})}$
- Double precision: 1 + 11 + 52 bits
- Limited precision: 0.1 is stored approximately
- Never test floats for exact equality

Data in Memory and Endianness

Store the 32-bit value 0x12345678

Address	Big-endian	Little-endian
0x1000	12	78
0x1001	34	56
0x1002	56	34
0x1003	78	12

Most significant byte first vs least significant byte first

Big-endian

MSB at the lowest address. Used in network byte order.

Little-endian

LSB at the lowest address. Used by x86 and typical ARM systems.

Alignment

A 4-byte value should start at an address divisible by 4

Text, Images and Sound as Bits

Aa

Text

Each character gets a code

ASCII: 'H' = 48, 'i' = 69 (hex). Unicode / UTF-8 covers all scripts.

RGB

Images

A grid of pixels

Each pixel = 3 bytes (R, G, B). 24 bits give 16.7 million colours.

♪

Sound

Samples of a wave

CD audio: 44,100 samples per second, 16 bits each, 2 channels.

A bit pattern has no meaning by itself: the program decides how to interpret it.

Binary Logic and Boolean Algebra

AND

$x \cdot y$ or xy

1 only if both inputs are 1

OR

$x + y$

1 if at least one input is 1

NOT

x'

Inverts the input

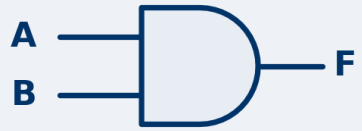
A Boolean function can be shown three ways

Truth table | algebraic expression | logic diagram

Careful with notation

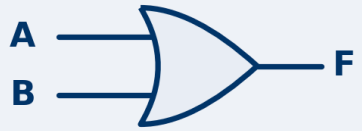
In Boolean algebra 1 + 1 = 1 (OR). In binary addition 1 + 1 = 10.

Logic Gates and Their Symbols



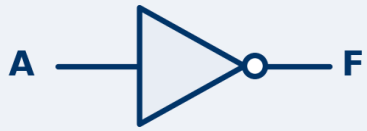
AND

1 if both inputs are 1



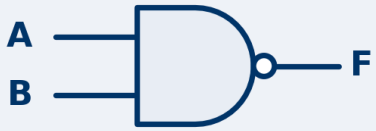
OR

1 if any input is 1



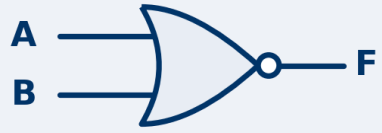
NOT

Inverts the input



NAND

NOT (AND)



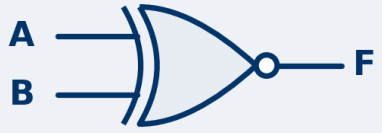
NOR

NOT (OR)



XOR

1 if inputs differ



XNOR

1 if inputs are equal

Remember

The bubble means inversion.

Truth Tables for All Gates

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

NOT
$0 \rightarrow 1$ and $1 \rightarrow 0$ (one input)

n inputs
2^n rows in the truth table

Half adder
Sum = A XOR B; Carry = A AND B

Boolean Algebra Laws

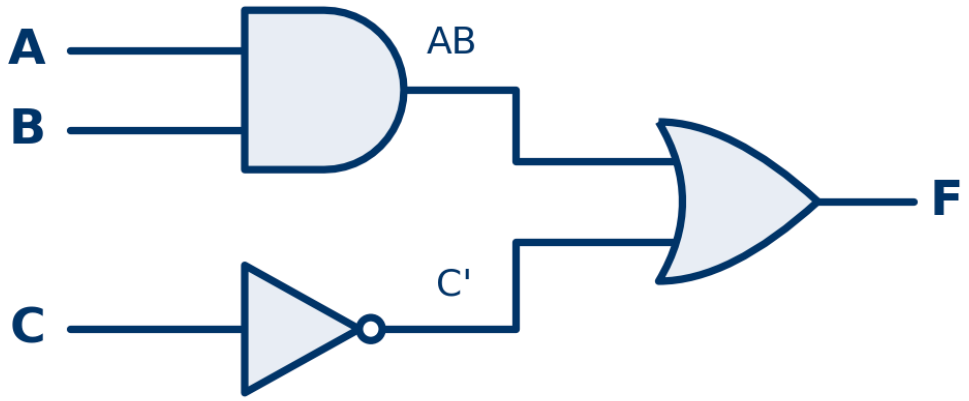
Law	AND form	OR form
Identity	$x \cdot 1 = x$	$x + 0 = x$
Null	$x \cdot 0 = 0$	$x + 1 = 1$
Idempotent	$x \cdot x = x$	$x + x = x$
Complement	$x \cdot x' = 0$	$x + x' = 1$
Distributive	$x(y + z) = xy + xz$	$x + yz = (x + y)(x + z)$
De Morgan	$(xy)' = x' + y'$	$(x + y)' = x'y'$
Absorption	$x(x + y) = x$	$x + xy = x$

Duality: swap AND with OR and 0 with 1 in any law to get its partner.

From Expression to Circuit

A	B	C	AB	C'	F
0	0	0	0	1	**1**
0	0	1	0	0	**0**
0	1	0	0	1	**1**
0	1	1	0	0	**0**
1	0	0	0	1	**1**
1	0	1	0	0	**0**
1	1	0	1	1	**1**
1	1	1	1	0	**1**

F = A·B + C'



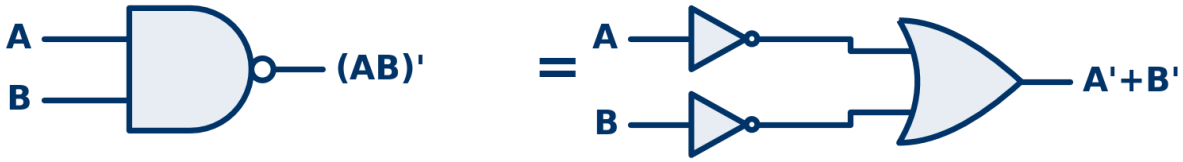
Build → verify: one gate per operator, then test all 8 input rows (lab activity)

Standard Forms and Universal Gates

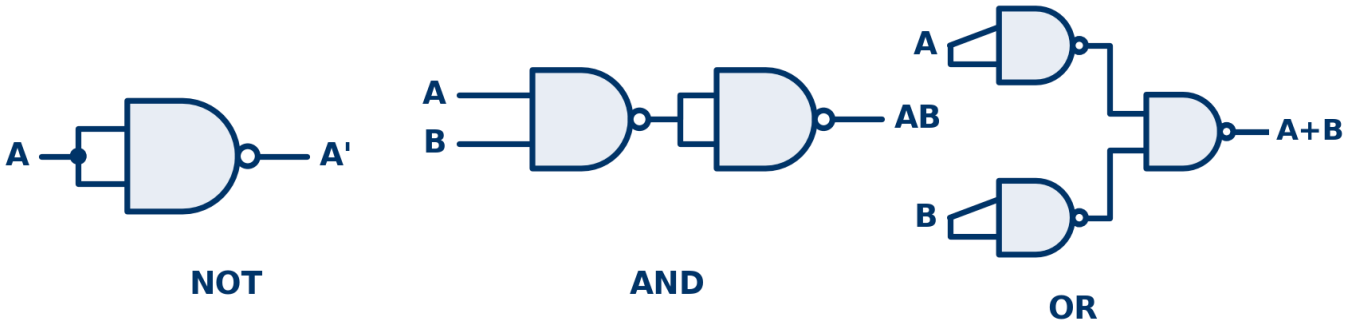
Canonical SOP

F = Σ m(0, 2, 4, 6, 7)

One minterm per row where F = 1



De Morgan: NAND = OR with inverted inputs



Summary and Bridge to the Lab

- Subtraction is addition of a 2's complement; overflow needs a check
- Signed numbers use 2's complement: range -2^{n-1} to $2^{n-1} - 1$
- Text, images, sound and reals are all bit patterns with an agreed code
- Boolean algebra describes gates; truth tables prove equivalence
- NAND (or NOR) alone can build any circuit

Questions?

Lab session

System overview, Linux setup, conversion exercises, logic gate simulation

Homework

Mano Ch. 1 and Ch. 2 problems: conversions, complements, truth tables

Reading

Mano §1-5 to 1-9 and §2-1 to 2-7; H&P §1.4 to 1.5